

Rails will have different values than those issued for GitLab's OIDC ID tokens.

mermaid
flowchart LR

glab[GitLab Rails]
o{OpenBao}
p[Pipeline]

glab --authenticated secrets management-->o

glab --issues JWT--> p --authenticated secrets fetch-->o

Importantly, while GitLab Rails authorizes the pipeline's execution of a particular job, the pipeline's token must be adequately scoped to a particular execution context. A pipeline's execution context is defined as follows:

mermaid
flowchart LR

u(Owner)
g(Groups)
sg(Group Namespaces)
o(Organization)
og(Org Groups)
sog(Org Namespaces)
proj(Project)
pln[Pipelines]
env[Environments]
branch[Branches]

u --contains--> proj
g -- optionally contains --> sg
sg --contains--> proj
g --contains--> proj
o --contains--> og
og -- optionally contains --> sog
sog --contains--> proj
og --contains--> proj
proj --contains--> pln
pln --contains--> env
pln --contains--> branch

Both groups and organization groups can contain many nested subgroups ("namespaces"). The existing OIDC ID token contains sufficient context information for these purposes.

From the context of an execution of a pipeline, a secret may come from many sources:

mermaid
flowchart LR

u(Owner)
g(Groups)
o(Organization)
og(Org Groups)
proj(Project)
pln[Pipeline]
env[Environments]
branch[Branches]
exec{Execution}

exec -- of a--> pln
exec --contains multiple--> branch --in the context of--> proj
exec --contains multiple--> env --in the context of--> proj
exec --in the context of--> proj

proj --in the context of --> og --in the context of --> o
proj --in the context of --> u
proj --in the context of --> g

Here, an execution of a pipeline can, in the restriction of a set of branch or environment match patterns (e.g., main, , or release/), can get secrets from:

1. A project,
1. The (user) owner of a project or equivalently,
1. The hierarchical groups up to and including the organization owner of a project, or a parent group (prior to the introduction of organizations).

That is, if a secret is to be used by multiple projects, it should be provisioned up a level and the branch and/or environment restricted appropriately, ensuring that the relevant environment does not conflict across projects it should not be visible in.

Similarly, in the context of user access and management of secrets, the following hierarchy applies:

mermaid
flowchart LR

u{User}

secret[Secret]
projTeam[Project Team]

groupTeam[Group Team]
orgTeam[Organization Team]

da(Direct Access)
sm(Secrets Management role)

u -- given direct access to --> secret
u -- belongs to --> projTeam
u -- belongs to --> groupTeam
u -- belongs to --> orgTeam

projTeam -- granted --> da
projTeam -- granted --> sm
groupTeam -- granted --> da
groupTeam -- granted --> sm
orgTeam -- granted --> da
orgTeam -- granted --> sm

A team at any level (project, group, or organization) can be given permission to manage all secrets at or below its level ("Secrets Management capability" above). However, on an individual secret, a user or team can also be given direct access even if they don't otherwise have secrets management capabilities.

Initially, users will manage secrets through GitLab Rails and its broader privileged JWT tokens, but the intent longer-term is to use per-user JWTs. The current design necessitates that GitLab Rails sees the value of the secret, proxying it on behalf of the Browser UI to OpenBao. By using a per-user JWT issued to the browser session in LocalStorage, the UI can directly contact the public OpenBao instance and GitLab Rails will not have access to the secret during provisioning and thus, risk inadvertently logging it. However, the privileged JWT token will still be used to safely limit changes to the ACL policies.

Secret and authentication hierarchy

Design of OpenBao's mount path will greatly affect available security parameters. When encoding user values ({value}), we'll use URL-safe Base64: this lets us have a unique, canonical transformation of potentially non-path-safe components into individual path segments. For components with dynamic names (like user, organization, and group names), which aren't glob-aware and which are subject to renaming, we'll use their internal integer database identifiers to prevent having to rename the underlying mounts.

Tenant definition

We assume every project has a parent component: this is either a user, a group, or an organization. In the event of legacy groups with orgid=1

we will use the owning group instead.

Secrets

We propose the following structure for mounts for user-owned projects:

- /user{userid}/secrets
- /user{userid}/proj{projectid}/secrets

And for organization-owned projects:

- /org{orgid}/secrets
- /org{orgid}/namespace{nsid}/secrets
- /org{orgid}/proj{projectid}/secrets

For groups which use the existing group-based system, the top-level entity would be the parent group:

- /group{groupid}/secrets
- /group{groupid}/namespace{nsid}/secrets
- /group{groupid}/proj{projectid}/secrets

Here, the first group{groupid} would be the top-most group-id, but would not be repeated for the nested sub-group secrets.

namespace{nsid} indicates a nested entity (whether a group, subgroup, or user namespace within a top-level organization or group). With the eventual introduction of top-level organizations, if groups belong to orgid=1 meta-org, we'll use group-based tenant separation for these.

Within each secrets folder, we'll initially mount a K/V secrets engine at /kv; eventually other types of secrets engines can also be mounted to provide dynamic secrets, with some consideration as to naming engines.

Within each K/V secrets engine, we'll use the following nesting structure to restrict access to secrets (with a given <name> in pipelines:

- .../secrets/kv/data/explicit/<name>

Similarly, when using dynamic secrets engines with roles, we'll suggest using the following structure of mounts and allow configuring secret names inside of these mounts as appropriate to the specified dynamic secret engine:

- .../secrets/<mount-name>/.../explicit-<role-name>

Succinctly, the requirement that any given path a secret could live at is prefix-free: we use explicit to indicate that provisioning auth engines must explicitly grant secret-level access to specific fields. If later types of secret protection schemes are used (such as placing environment or branch protections in the URL to allow broad access to any secret for that branch

or environment), we suggest using env- and branch- as prefixes.

Authentication

Authentication uses five sets of mounts:

- /auth/user{userid}/pipelinejwt
- /auth/group{groupid}/pipelinejwt
- /auth/org{orgid}/pipelinejwt
- /auth/userjwt
- /auth/gitlabjwt

In particular, because pipelines may need access to nested secrets, but won't need access to anything outside the tenant's scope, we will provision ACL policies and authentication at the top-most tenant namespace (user, group or org). With namespaces, this will help with restricting the pipeline from escaping its sandbox and accessing other tenant's secrets; however, we rely on careful ACLs to protect access to other projects and secrets within the tenant namespace.

Later, user authentication can be added to each tenant (/auth/org{orgid}/jwtuser or /auth/group{groupid}/jwtuser) and the user can request scoped JWTs (and subsequent OpenBao tokens) so that they aren't requesting tokens with broad access beyond the scope they are modifying the secrets of.

Aside: presently the order is auth and then tenant segment, but when adding proper namespace support, auth mounts could be inside of a tenant and thus the order will be swapped to e.g., /user{userid}/auth/pipelinejwt.

GitLab Privileged JWT

Initially using the same JWT issuer, though potentially migrating to a purpose-built issuer, GitLab will issue itself JWTs to authenticate against the privileged /auth/gitlabjwt path. The subject of this token and other fields will be uniquely different than the OIDC ID tokens issued to pipelines, preventing them from successfully authenticating against this endpoint.

Long-term, we'll attempt to tie inbound GitLab Rails requests to outbound, user-initiated management actions via custom claims on these JWTs, allowing auditing and attribution from source request through to OpenBao.

Using JWTs instead of AppRole removes the need for additional, external secrets management in the operational side of GitLab Rails: it already has access to the provisioned JWT issuer and thus can self-issue JWTs to auth against OpenBao as it remains in a privileged place in the design.

ACL design

We propose a group-based approach to ACL policies: each scope (for a pipeline)

and role (for users, when not using explicit grants) will grant access to specific subsets of secrets. These policies are maintained and stored in OpenBao, but GitLab Rails is tasked with managing and provisioning them.

Notably, no Rails-initiated operations are expected to span multiple tenant contexts. This allows us to add per-namespace ACLs in the future and create smaller path->policy indices in the future. Furthermore, we can use nested paths to segment different policies and build per-segment indices, reducing the list operation overhead as well.

For each policy, we'll present create a group alias to allow a groupsclaim on the Rails-issued JWT to select applicable ACL policies based. A future enhancement will allow us to get rid of all but glob-based group matches.

Hierarchy of policies

Policies will be prefixed by their tenant information (e.g., user{id}/) until they are migrated to proper namespaces.

For each project, we'll provision ACL policies prefixed with project{id}/: this path separator component is an allowed character in policy names and allows us to use a future extension to list just policies we are interested in and will let us reduce the size (and increase the relevance) of indices.

Additionally in the future, hierarchical secrets can be supported by also supporting group{id} and other constructs as top-level categories. With multi-tenant support, we'll have fewer top-level items (as they'll be explicitly bounded by the tenant), making iterating over all such items easier. However, we'll usually have fairly few items.

Note that these do not necessary reflect the secret's path and only notate where ACL policies exist. Because tenant information will eventually be conveyed within a namespace (and the policies moved appropriately), we'll eventually end up with a secret-like ACL policy hierarchy.

Each top-level segment essentially represents all access to a particular secrets management section of the UI: project{id}/, group{id}/ &c.

Pipeline ACL

To restrict a pipeline's JWT token to only allowed paths, we'll use GitLab Rail to provision a just-in-time ACL policy for the pipeline and a JWT role tightly scoping to the expected claims from the existing CI/CD OIDC ID token integration.

OpenBao's ACLs are directly stored on disk, with a moderate cache for recently-used policies. Policies are loaded and evaluated at

request validation time,
meaning that if a pipeline ACL name is modified and reused, the latest
version (at the time of the request) will win.

Likewise, the JWT auth method's login
will require a role parameter to select the correct role for authentication.

For naming roles, we suggest the format:

- project{projid}

Layout

Within the project{id}/ top-level path segment, we'll provision an
additional path segment, pipelines/, to separate pipeline-related
policies for a project from other types of access.

We create the following types of pipeline policies with Rails:

- global: secrets any pipeline can access
- env/{context}: environment restriction policies
- branch/{context}: branch restriction policies
- combined/env/{context}/branch/{context}: combined environment and branch restrictions (AND)

A full path of an ACL would thus look like the following examples:

- project12345/pipelines/global,
- project12345/pipelines/env/prod-,
- project12345/pipelines/branch/release/, or
- project12345/pipelines/combined/env/prod-/branch/release/.

Notably, the direct encoding of restriction to path allows for us to create
groups with the same encoding, reducing the
need for GitLab Rails to query the set of restrictions before issuing the
JWT. Use of combined as a prefix ensures we cannot have confusion attacks
with poorly named environments and branches.

In the future, explicit grants could also occur: stages could have a path
segment and policies (stages/{name}/global) and each job in a stage could
also have direct secret access (stages/{name}/job/{name}/global). Or, we
could even support ANDing between stage, name, and the above restrictions
(environment/branch) to support rather granular execution contexts for these
jobs.

When issuing a JWT, presently GitLab Rails will need to query relevant
ACLs within a path and issue a groupsclaim field
with all the relevant glob values from the ACL list. However, with the
mentioned glob enhancements, GitLab Rails should be able to directly compute
these without requiring a lookup from OpenBao as this information already
appears on the idtoken.

The one exception is that the future enhancement for direct explicit grant (by stage/job name) does not yet exist on the id token and thus cannot be used for ACLing. However, this can easily be added as the information exists on the Ci::Build object (stage and name fields).

Contents

Each ACL policy will be an explicit grant of capabilities over a path. We will set these with GitLab Rails and will not use globbing (+ or *) to grant access to multiple secrets; only access to specific secrets will be allowed.

For example, if any pipeline running with an env/prod-<DATE> context is to have access to the production database credentials, we will create a policy named user12345/project54321/pipelines/env/prod- with the contents:

```
hcl
path "user12345/project54321/secrets/kv/data/explicit/PRODDBPASS" {
  capabilities = [ "read" ]
}
```

Notably, because a pipeline will have multiple contexts which might provision different ACL policies, we'll eventually want to implement something similar to policy unions.

User ACL

While the user wouldn't initially contact OpenBao directly (and instead have secrets management operations proxied through GitLab Rails), we want to switch GitLab Rails to using scoped user-level JWTs and then ultimately to the client having direct access to OpenBao (without necessarily contacting GitLab Rails), so that no user-confusion attacks are viable and so that GitLab Rails does not see the secrets during provisioning, preventing them from accidentally being logged in the future.

Notably, a user token would be rather privileged: it would have complete access to all secrets within the users' purview and thus should be suitably short-lived to avoid potential side-channel attacks. Further hardening (like auto-binding to CIDRs) would help avoid these issues. To solve this, we'd need to scope access along team lines (e.g., any teams with access to manage a certain category of secrets would have a separate ACL policy) along with any location-specific secrets granted in a given view and expand OpenBao to allow templating of profiles (so that the JWT could contain a concrete list of profiles to grant this user). This would be rather involved but would help scope user JWTs more specifically.

The upgrade path (from privileged JWT token to incrementally using tightly scoped policies) would simply be changing the Rails code to generate a user

JWT and updating the Ruby client's request to authenticate and use the subsequent token for a particular request instead of the privileged JWT token.

Note that users lack read permissions on the actual secret; they can only set the value. When rolling out dynamic secrets, users would be granted full permissions on the role, but no permissions on the dynamic secret endpoint (e.g., for PKI, they could have full permissions on `pki/roles/:name`, but none on `pki/issue/:name`).

Layout

Similar to pipelines, we'll create policies under the relevant top-level path, with a `roles/` or `direct/` subcomponent. Here, `roles/` will have various default or custom roles which get access to secrets at various levels. For instance, `roles/maintainers` and `roles/owners`: this allows each project to control what scope of access these roles get, if the defaults are not ideal.

Further, with `direct/user{id}` roles, users will be given explicit grants to certain secrets, beyond what they might normally see given their role.

Contents

For both roles and direct access, there are two types of grants:

1. Broad grants to the entire category of secrets.
2. Specific grants to individual secrets.

For the former, the policies might look like:

```
hcl
path "org{orgid}/project{projectid}/secrets/kv/data/explicit/+" {
  capabilities = [ "sudo", "create", "update", "patch", "delete", "list" ]
}
```

for read-write access and

```
hcl
path "org{orgid}/project{projectid}/secrets/kv/data/explicit/+" {
  capabilities = [ "list" ]
}
```

for view-only access.

For the latter (specific grants), the policies might look like the above, just with explicit names in them (e.g., `DBPASSPROD`).

When accessing a secrets management page, GitLab Rails will issue a JWT

to the user which will contain the relevant groupsclaims to groups with specific policies within the project. Notably, this will not delay load: this token will only be used by the user to set specific secrets, though a similar JWT and secret could be used on the GitLab Rails' backend to render the initial page. The assumption here being the Rails->OpenBao interconnect is faster than User->OpenBao and potentially Rails could have caching of user or secret lists.

Modifying ACL policies

Notably, ACL policies are written in HCL: this presents a problem since Ruby lacks good HCL support. However, HCL is JSON-compatible.

This means that, for any given ACL policy above, we can construct its equivalent in JSON, giving GitLab Rails native capabilities to edit it without pulling in another dependency or building our own parser.

For example, the policy in HCL:

```
hcl
path "org{orgid}/project{projectid}/secrets/kv/data/explicit/+" {
  capabilities = [ "list" ]
}
```

would be equivalent to the following JSON:

```
json
{
  "path": {
    "org{orgid}/project{projectid}/secrets/kv/data/explicit/+": {
      "capabilities": [
        "list"
      ]
    }
  }
}
```

This becomes much easier for GitLab Rails to query and update.

Static vs Dynamic Secrets

GitLab Secrets Manager will initially only support static secrets, stored in a KVv2 engine. Some improvements to OpenBao can be added to static secrets, such as:

1. Using leases, to support one-time use secrets. These could be written by another workflow and removed on use.

1. Supporting automatic deletion of secrets: a given expiration date could be strictly enforced, preventing use of the secret and requiring rotation of even long-lived manual secrets.

However, ultimately automatic rotation is an important consideration. By using fully dynamic secrets, the problem of rotation is avoided: secrets are scoped to a particular access and are automatically revoked (through lease expiration or revocation) once the pipeline or access finishes. This prevents reliance on static values and ensures compromise of values require time-limited use and prevent persistence. We can build several improvements to OpenBao to help the adoption of dynamic secrets:

1. Add a plugin registry to allow custom, external plugins or allow containerized execution of plugins.
1. Create additional backend-pluggable plugin types beyond database: this could allow various access-token providers to be more easily created by exposing common APIs in a standardized format.

We ultimately wish to push more users towards dynamic secrets. These are more secure and don't have long-term data retention and security concerns.

Sources of Truth

GitLab Rails is the source of truth for:

- JWT issuance via OIDC
- Pipeline issuance
- User-level issuance (eventually)

OpenBao is the source of truth for:

- Definitions of secrets
 - Canonical locations inferable from the pipeline definition and contextual information like project, namespace, &c.
 - Ownership information (direct access, role access, &c)
 - Scope of pipeline access (environments, branches, &c)
 - Any textual description of the secret available to operators
- Values of secrets
- ACL policies; these will be pushed from GitLab Rails based on ownership information (just before usage in the case of pipelines).
- Configuration of dynamic secrets, including K/V extensions for one-time use secrets, and automatic deletion.

GitLab Rails

GitLab Rails would be the main interface that users would interact with when managing secrets using the Secrets Manager feature. We would not provision direct user access to the underlying OpenBao instance and thus OpenBao's lack of a UI would not impact us: we wish to provide a more native, integrated UI than redirection

through OpenBao's UI would provide.

This component is a facade to OpenBao server, initiating several requests and workflows to OpenBao:

mermaid
flowchart LR

gl[GitLab Rails]
u{{User}}
p(Pipeline)
ob{OpenBao}

```
gl -- CUD secret (no Read) --> ob
gl -- manage mounts --> ob
gl -- manage auth roles --> ob
gl -- manage ACLs --> ob
gl-. issue user JWT .->u-. CUD secret (no Read) .->ob
gl -- issue pipeline JWT --> p
```

```
p -- JWT auth --> ob
p -- fetch secrets --> ob
```

```
ob -- audit --> gl
```

Initially all operations will through GitLab Rails including CUD operations on secrets, but not requests for secrets from pipelines (hence, excluding the R in CRUD -- though this action is of course initiated by a pipeline job started from, and with a JWT issued by, GitLab Rails). Eventually the privileged token for CUD operation will be replaced with a user JWT and subsequently directly by the user (through a JavaScript OpenBao client running in the context of the GitLab UI).

Within the GitLab Rails monolith, two components will need access to the secrets manager: Puma, where the majority of the GitLab backend code executes; and Sidekiq, a background job processor to allow long-lived background operations (such as initial provisioning of OpenBao and secret engine mounts).

Management of secrets

Because OpenBao remains the source of truth for many decisions, GitLab can retain only a minimum table indicating the enablement status of secrets on projects.

In SQL, this could look like:

```

sql
CREATE TABLE secrets (
  id bigint NOT NULL,

  // Scope information; only one is set. Currently only project-level
  // secrets are allowed.
  // userid bigint,
  // organizationid bigint,
  // groupid bigint,
  projectid bigint,

  // enablement status: provisioning, active, or disabled.
  status statemachine,
);

```

Here, groupid references the closest-most subgroup that owns the secret instance. Parent subgroups and the namespace can then be queried.

Management of authentication

GitLab Rails needs to issue two types of JWTs:

1. JWTs within the scope of a project for a particular pipeline execution.
1. JWTs for user authentication.

GitLab already supports issuance of both JWT types through its OIDC for CI/CD

and as an Identity Provider (IDP).

Within each auth method, we'll need to ensure that we create the necessary roles to ensure OIDC tokens get exchanged for OpenBao tokens with appropriate JWT-specific policies attached. As discussed in ADR-008: Redesigning secrets manager without a Rails database table, this will require simple extensions to OpenBao to do.

Necessary cleanup hooks

The following places need to be hooked to update secrets (both internally to the database and in OpenBao):

- Deletion of project
- Deletion of group
- Deletion of organization
- Deletion of user
- Cancellation of a pipeline
- Disabling of secrets on a project
- Non-renewal of secrets management subscription

Because IDs are used in place of names, few mount move operations will be incurred except when enabling multi-tenant namespaces for the first time.

OpenBao server

OpenBao server will be a new component in the GitLab overall architecture. This component provides all the secrets management capabilities including storing the secrets themselves. An overview of its internals and threat model are available on its project website.

In summary, storage is encrypted with a root key, which in turn encrypts a barrier keyring, which in turn encrypts storage. Access to the root key is protected through either a Shamir's derived AES key (which requires a quorum present to unseal) or an automatic unseal mechanism (such as GCP KMS).

Storage backend

OpenBao on Runway will initially be deployed with the Postgres engine. This is a sane default for self-hosted as well, as GitLab Rails already requires a database and can create tables in it. However, support for the more widely deployed Raft backend can also be added in the future fairly easily.

Seal mechanisms

For GitLab.Com deployments, OpenBao will use the GCP KMS auto-unseal mechanism. For self-hosted environments, if an existing supported KMS mechanism is not provided, we can use the Shamir seal for MVC. Later improvements might include tying into the existing secrets file as an auto-unseal mechanism for OpenBao, supporting PKCS11 seals, and multi-unseal capabilities (discussed below).

Audit logging

OpenBao supports audit logging through file, syslog, or socket mechanisms. Because OpenBao will not necessarily be co-located with GitLab Rails or our audit logging (into Postgres), it is suggested to either use the syslog backend or use a custom audit logging backend.

Note that, in the event audit logging goes down, OpenBao will not be able to handle any requests until this is fixed.

High availability

GitLab Rails includes Geo support for self-hosted clustering, allowing multiple sites to exist, each with its own set of nodes. Each site has its own PostgreSQL replica. One site is designated primary and nodes in the site are allowed to perform write operations to the Postgres database. All PostgreSQL data is available on all sites. While GitLab has selective syncing of repository data, projects

are globally accessible from all sites and nodes. While currently only secondary nodes talk to the primary site, this should be a bidirectional channel.

OpenBao natively includes High Availability support; this is either provided by Raft or the PostgreSQL storage backends. The latter uses a lock table to implement leadership election. Only one node is marked active at a time, with secondary nodes not performing any operations except forwarding requests to the primary.

Notably, the semantics of Geo and OpenBao roughly align. We propose that Geo will need no additional enhancements to support GitLab Secrets Manager and that replication will be handled by the latter when using Raft.

On all front-end service nodes, we'll start the self-hosted OpenBao server instance. One node will be designated primary by OpenBao HA election: initially this will be a random node, but in the future we could let Geo inform OpenBao which site is designated primary and the leader election process could be changed. This node will use OpenBao's native HA capabilities: standby nodes will proxy all operations (initially, later serving read requests) to the active OpenBao instance.

With the Raft storage backend, each front-end node will have local storage it can use for placing Raft's underlying bbolt K/V store. In the event of an even number of nodes in the primary site, we will proactively designate one node to be a non-voter node. From Geo's information, we'll populate all node's retryjoin configurations with reference to the other nodes for discoverability. In the future, we can also designate non-primary sites to be non-voter nodes as well. The number of sites or latency of replication will thus not impact the latency of writes in the general case.

With the PostgreSQL storage backend, we can rely on Geo's existing replication of the PostgreSQL backend and no additional changes will be necessary.

When runners contact the OpenBao instance, if their request does not hit the active node, OpenBao will route the request through its GRPC request forwarding mechanism.

In the event of a failover, Geo will be able to bring up the new site designated as primary and data will already have been replicated, either through PostgreSQL's replication or through Raft's synchronization process. In the future and in the case of the latter, Rails, via Geo's indication, will update the node's Raft configuration to no longer be non-voter and restart the node so a new leader is elected. In the event of later improvements to Postgres backend to indicate desired leadership status, a similar change could be applied there as well when a site's status changes. This will also help to align the definitions of primary sites between Geo and OpenBao.

The net result is that Geo is not responsible for data replication for OpenBao, but is still used as a source of leadership data so that a consistent customer experience is achieved.

Cells and multi-replication zones

Initially we will support one logical OpenBao cluster per instance. In the future with cells, we'd expect each tenant to cluster affinity:

Required enhancements

OpenBao will require enhancements to improve security prior to GA of this feature. These are discussed more below.

Multi-unseal capabilities

In the event an unseal method has faulted and is unrecoverable, OpenBao today will be unable to decrypt its secrets and thus fail to start. In the event of infrastructure outages, this makes recovery hard. Upstream Vault Enterprise has introduced Seal HA capabilities, whose design was influenced by the additional requirements of Seal Wrapping.

OpenBao does not have Seal Wrapping and thus can rely on a design similar to an earlier, retracted PR.

We can allow several parallel unseal mechanisms, each of which encrypts the same root key. This would allow a mix of auto-unseal and Shamir's, allowing a hybrid recovery procedure in the event the auto-unseal mechanism were unavailable or the key was lost. Notably, reuse of the root key isn't bad: any loss of unseal key material (that compromises root key integrity) with separate root keys would still allow recovery of the barrier encryption key. Rotation of the root key would require all unseal mechanisms to either be online or removed prior to use. However, we wouldn't have to contend with secrets encrypted using a combination of mechanisms, as they only encrypted entry is the root key.

Multi-tenancy through namespaces

In the current design document, separate tenants' secrets would be encrypted using the same barrier encryption keys. This means compromise of a single tenant would result in compromise of all tenants.

OpenBao lacks namespaces from Vault Enterprise.

These allow for logical separation of tenants, as discussed upstream. In particular, we can grant limited, scoped sys/ capabilities to admins within a namespaces:

- sys/auth to mount new auth plugins.
- sys/mounts to mount new secret engine plugins.

- sys/namespaces to allow nested namespaces.
- sys/policies to control namespace-restricted policies.
- Optionally sys/quotas/config and sys/quotas/rate-limit to control namespace-restricted quotas (if tenants want to set their own, subject to global quota limits).

The intention of this change would be to safely allow customers to directly interact with their OpenBao instances, bringing non-Pipeline workloads to use the same secrets store.

Further, we can add layered seal encryption to namespaces, creating a separate root key and barrier keyring encrypted by both the parent namespace's barrier key but also a separate unseal mechanism. This would likely expose the following scoped sys/ routes within a namespace:

- sys/rotate to rotate the per-namespace barrier encryption key.
- sys/rekey and sys/rekey-recovery-key to rotate the per-namespace root and recovery keys (if any).
- sys/seal and sys/unseal to seal and unseal the namespace. Note that sealing an auto-unseal mount would require the recovery keys.

In the event a customer wanted to revoke access to secrets from GitLab, they could seal their namespace and prevent GitLab from accessing the unseal mechanism of choice.

Lastly, we can strengthen the memory isolation of customers by only allowing plugin multiplexing within a namespace. When coupled with external runners for plugins, such as a container or cgroups, we could further isolate tenants data in memory.

Chosen leader and Raft updates

When working in a Geo cluster, we'd ideally like the OpenBao primary node to align with the Geo cluster's definition of the primary site. We'll want update OpenBao to give a suggested leader or add non-leader/voter status to nodes on secondary sites.

Packaging and deployment

OpenBao maintains an upstream Helm chart that can be used for deploying OpenBao in a Kubernetes environment. This can be referenced and configured from the GitLab Helm chart as required.

For self-hosted, OpenBao server will also be executed by GitLab Rails.

Use case studies

- Using secrets in a CI job

Further investigations required

1. Authentication of clients other than GitLab Runner.

GitLab Runner authenticates using JWT, for other types of clients, we need a secure and reliable method to authenticate requests to decrypt a secret.

1. Infrastructure setup, including how OpenBao will be installed for self-managed instances.

1. How to best implement sharing of secrets between multiple groups in GitLab.

1. Establish our protocol and processes for incidents that may require sealing the secrets vault.

1. How to support protected and environment specific rules for secrets.

1. How to audit secret changes. Do we want to use audit socket?

1. Should the secrets be revoked if a project or subgroup is moved under a different top-level group/organization?

Alternative solutions

Other solutions we have explored:

- Separating secrets from CI/CD variables as a separate model with limited access, to avoid unintended exposure of the secret.
- Secure Files
- Implementing secrets management from scratch (see Superseded).

References

The following links provide additional information that may be relevant to secret management concepts.

- OWASP Secrets Management Cheat Sheet
- OWASP Key Management Cheat Sheet

Who

DRIs:

<!-- vale gitlab.Spelling = NO -->

Role	Who	
Author	Erick Bajao, Senior Engineer	
Co-Author	Alex Scheel, Staff Engineer	
Recommender	Fabio Pitino, Principal Engineer	
Product Leadership	Jocelyn Eillis, Product Manager	
Engineering Leadership	Scott Hampton, Engineering Manager	
Lead Engineer	Erick Bajao, Senior Backend Engineer	
Senior Engineer	Maxime Orefice, Senior Backend Engineer	
Engineer	Shabini Rajadas, Backend Engineer	

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-
documents/secret_manager/decisions/001_envelop_encryption.md

Context

To store secrets securely in the GitLab Secrets Manager, we need a system that can prevent unencrypted secrets from being leaked in the event of a security breach to a GitLab system.

Decision

Use envelope encryption. GitLab Rails will store the encrypted secret at rest, along with an encrypted data key.

In order to decrypt the secret, GitLab Rails will need to make a decryption request to the GCP key manager through GitLab

Secrets Service and obtain the decrypted data key. The data key is then used to decrypt the encrypted secret.

mermaid

sequenceDiagram

participant A as Client

participant B as GitLab Rails

participant C as GitLab Secrets Service

Note over B,C: Initialize vault for project/group/organization

B->>C: Initialize vault - create key pair

C->>B: Returns vault public key

B->>B: Stores vault public key

Note over A,C: Creating a new secret

A->>B: Create new secret

B->>B: Generate new symmetric data key

B->>B: Encrypts secret with data key

B->>B: Encrypts data key with vault public key

B->>B: Stores envelope (encrypted secret + encrypted data key)

B-->>B: Discards plain-text data key

B->>A: Success

Note over A,C: Retrieving a secret

A->>B: Get secret

B->>B: Retrieves envelope (encrypted secret + encrypted data key)

B->>C: Decrypt data key

C->>C: Decrypt data key using vault private key

C->>B: Returns plain-text data key

B->>B: Decrypts secret

B-->>B: Discards plain-text data key

B->>A: Returns secret

Consequences

With this approach, an actor that gains access to the GitLab database containing the envelope will not be able to decrypt the content of the secret as the private key required is not stored with it.

We also need to consider how to securely generate and store the asymmetric keypair used for each vault.

In addition, the following resources would be required:

1. Multiple asymmetric keypairs. A unique asymmetric keypair is needed per vault, belonging to a project, group or an organization.
1. Multiple symmetric keys. A unique key is needed per secret.

Alternatives

We considered performing the encryption and decryption of the secret in the GitLab Secrets Service, while storing the encrypted data in GitLab Rails. However, this means that there would be a time where the secret and the encryption keys exist at the same time in GitLab Secrets Service.

SECTION: engineering/architecture/design-documents/secret_manager/decisions/002_gcp_kms.md

Context

Following from ADR 001: Use envelope encryption, we need to find a solution to securely store asymmetric keys belonging to each vault.

Decision

We decided to rely on Google Cloud Platform (GCP) Key Management Service (KMS) to manage the asymmetric keys used by the GitLab Secrets Manager vaults.

Using GCP provides a few advantages:

1. Avoid implementing our own secure storage of cryptographic keys.
1. Support for Hardware Security Modules (HSM).

mermaid

sequenceDiagram

participant A as Client

participant B as GitLab Rails

participant C as GitLab Secrets Service

participant D as GCP Key Management Service

Note over B,D: Initialize vault for project/group/organization

B->>C: Initialize vault - create key pair

Note over D: Incurs cost per key

C->>D: Create new asymmetric key

D->>C: Returns public key

C->>B: Returns vault public key

B->>B: Stores vault public key

Note over A,C: Creating a new secret

A->>B: Create new secret

B->>B: Generate new symmetric data key

B->>B: Encrypts secret with data key

B->>B: Encrypts data key with vault public key

B->>B: Stores envelope (encrypted secret + encrypted data key)

B->>B: Discards plain-text data key

B->>A: Success

Note over A,D: Retrieving a secret

A->>B: Get secret

B->>B: Retrieves envelope (encrypted secret + encrypted data key)

B->>C: Decrypt data key

Note over D: Incurs cost per decryption request

C->>D: Decrypt data key

D->>C: Returns plain-text data key

C->>B: Returns plain-text data key

B->>B: Decrypts secret

B->>B: Discards plain-text data key

B->>A: Returns secret

For security purpose, we decided to use Hardware Security Module (HSM) to protect the keys in GCP KMS.

Consequences

Authentication

With keys stored in GCP KMS, we need to de-multiplex between identities configured in GCP KMS and identities defined in GitLab so that decryption requests can be authenticated accordingly.

Cost

With the use of GCP KMS, we need to account for the following cost:

1. Number of keys required

1. Number of key operations
1. HSM Protection level

The number of keys required would be dependent on the number of projects, groups, and organizations using this feature.

A single asymmetric key is required for each project, group or organization.

Each cryptographic key operation would also incur cost and it varies per protection level. Based on the proposed design above, this would incur cost at each secret decryption request.

We may implement a multi-tier protection level, supporting different protection types for different users.

The pricing table of GCP KMS can be found [here](#).

Feature availability for Self-Managed customers

Using GCP KMS as a backend means that this solution cannot be deployed into self-managed environments.

To make this feature available to Self-Managed customers, this feature needs to be a GitLab Cloud Connector feature.

Alternatives

We considered generating and storing private keys within GitLab Secrets Service, but this would not meet the requirements for FIPS Compliance.

On the other hand, GCP HSM Keys comply with FIPS 140-2 Level 3.

SECTION: engineering/architecture/design-
documents/secret_manager/decisions/003_go_service.md

Following ADR-002 highlighting the need to integrate with GCP services, we do need to decide what tech stack is going to be used to build GitLab Secrets Manager Service (GSMS).

Context

At GitLab, we usually build satellite services around GitLab Rails in Go. This is especially a good choice of technology for services that may heavily leverage concurrency and caching, where cache could be invalidated / refreshed asynchronously.

Go-based GCP KMS
client library also seems to expose a reliable interface to access KMS.

Decision

Implement GitLab Secrets Manager Service in Go. Use labkit as a minimalist library to provide common functionality shared by satellite services.

Consequences

The team that is going to own GitLab Secrets Manager feature will need to gain more Go expertise.

Alternatives

We considered implementing GitLab Secrets Manager Service in Ruby, but we concluded that using Ruby will not allow us to build a service that will be efficient enough.

SECTION: engineering/architecture/design-
documents/secret_manager/decisions/004_stateless_kms.md

In ADR-002 we decided that we want to use Google's Cloud Key Management Service to store private encryption keys. This will allow us to meet various compliance requirements easier.

In this ADR we are going to describe the desired architecture of GitLab Secrets Management Service, making it a stateless service, that is not connected to a persistent datastore, other than an ephemeral local storage.

Context

Decision

Make GitLab Secrets Management Service a stateless application, not being connected to a global data storage, like a relational or NoSQL database.

We are only going to support local block storage, presumably only for caching purposes.

In order to manage decryption cost wisely, we would need to implement multi-tier protection layers, and in-memory, per-instance, symmetric decryption key caching, with cache TTL depending on the protection tier. A hardware or software key can be used in Google's Cloud KMS, depending on the tier too.

Consequences

1. All private keys are going to be stored in Google's Cloud KMS.
1. Multi-tier protection will be implemented, with higher tiers offering more protection.
1. Protection tier will be defined on per-organization level on the GitLab Rails Service side.
1. Depending on the protection level used, symmetric decryption keys can be in-memory cached.

1. The symmetric key's cache must not be valid for more than 24 hours..
1. The highest protection tier will use Hardware Security Module and no caching.
1. The GitLab Secrets Management Service will not store access-control metadata.
1. Identity de-multiplexing will happen on GitLab Rails Service side.
1. Decryption request will be signed by an organization's public key.
1. The service will verify decryption requestor's identity by checking the signature.

Alternatives

We considered using a relational database, or a NoSQL database, both self-managed and managed by a Cloud Provider, but concluded that this would add a lot of complexity and would weaken the security posture of the service.

SECTION: engineering/architecture/design-
documents/secret_manager/decisions/005_secrets_key_structure.md

Context

In GitLab, we have a hierarchical structure for projects and their parent namespaces wherein names can be identical in certain parts of the paths. We want to ensure that there are no conflicts with secrets paths across the hierarchy and across all customers when we store them in OpenBao.

Decision

While secrets are defined in a hierarchical fashion in the GitLab UI, the secret key paths are structured in a flat manner.

Consider the following example path of a project with nested namespaces:

- gitlab-org/ci-cd/verify/test-project
 - The secrets for the top-level group gitlab-org are stored under kv-v2/data/namespaces/ci/<ID of gitlab-org>
 - The secrets for the subgroup verify are stored under kv-v2/data/namespaces/ci/<ID of verify>
 - The secrets for the project test-project are stored under kv-v2/data/projects/ci/<ID of test-project>
 - Note the use of ci/ prefix so that we can group different types of secrets.

SECTION: engineering/architecture/design-
documents/secret_manager/decisions/006_approle_authentication_rails.md

Context

Given GitLab Rails acts as a facade over OpenBao, it directly communicates with the OpenBao API on behalf of users. This means the GitLab Rails component will be granted access to read and

write secrets and related configuration data for all projects and organizations to and from OpenBao. We must ensure that every request made by Rails to OpenBao is securely authenticated while staying performant.

Authentication options

We evaluated 2 OpenBao authentication methods for GitLab Rails:

1. JWT auth method

- This is similar to how Runner authenticates with OpenBao when fetching secrets in a CI job. In every request, Rails needs to generate an ID token to authenticate with OpenBao.

1. AppRole auth method

- Rails authenticates with OpenBao by providing a Role ID and a Secret ID which are already predefined.

Decision

With performance in mind, avoiding the extra steps that need to happen if Rails were to authenticate via JWT in each request, we decided to go with the AppRole authentication method.

Consequences

GitLab Rails is considered trusted environment, thus granting it access to manipulate secrets and related configuration across all organizations and projects in GitLab. Extra caution is needed to ensure that the secret ID for the role is protected and will never leak. Fortunately, AppRole usage best practices are documented.

SECTION: engineering/architecture/design-documents/secret_manager/decisions/007_openbao.md

Context

To store and maintain secrets securely in the GitLab Secrets Manager, we want to rely on a robust system that can provide the necessary features that we need.

Decision

Use OpenBao, a fork of HashiCorp Vault, as the secrets management service. This component will provide all the mechanism to securely store and manage secrets. In terms of user-initiated modifications of secrets, GitLab Rails will act as an abstraction layer and will delegate all tasks to this component.

Using OpenBao provides a few advantages:

1. Avoid implementing our own secure storage of secrets.
1. Support for Hardware Security Modules (HSM).
1. Leverage existing integration mechanism that we have for HashiCorp Vault because OpenBao maintains backwards compatibility with the open source edition of Vault.

Consequences

To provide uninterrupted access to secrets, we need the OpenBao vault to always be unsealed.

We have to ensure that the proper policies and access rights are in place to prevent actors from obtaining secrets in an event that they gain access to the container running GitLab Rails.

Also, given the encryption, decryption, and storage of secrets all happen in the OpenBao server, we have to make sure to harden the security and prevent a breach of the vault instance.

SECTION: engineering/architecture/design-
documents/secret_manager/decisions/008_no_database.md

One major issue with the database design for Tanukey is that it requires data synchronization across two distinct services, preventing the use of a common transaction. Using an external storage operation with OpenBao is impossible, so our next best design option is to use OpenBao as the single source of truth and identify what enhancements need to be made.

Our design goals are as such:

1. Strictly rely on OpenBao as much as possible and avoid database synchronization issues;
2. Minimize net-new enhancements to OpenBao;
3. Be cognizant of OpenBao's performance characteristics.

Why

In various discussions,

we've been realizing that the current design has lead to a potential for split-brain problems: creating or deleting a secret requires both a write to OpenBao and a write to a PostgreSQL table (through Rails/Puma). If either fails, a reconcile step would be necessary. Some possible designs included using a background job to retry provisioning the secret or the database table entry (which would potentially mean spreading that secret beyond OpenBao into the runner's queue), or using a later reconciliation background job (which could be expensive as you'd have to list all secrets in OpenBao to identify which ones lacked database entries).

Barring an easy way of solving the split brain problems means that we wanted to explore the impacts of a single-source-of-truth using OpenBao.

Summary

With careful layout of secrets, it should be possible for MVC to only rely on OpenBao as the single source of truth. For post-GA, there are definitely some future improvements that would be necessary for improved performance in

certain operations, but many of these can be solved by first (re-)adding horizontal scalability via servicing read-only requests on standby HA nodes.

Description

The below design relies on property-based authentication: GitLab Rails understands the context ("properties") of a request (whether from a user, such as roles or identifiers; or from a pipeline, such as environment, branch, and potentially including stage and job name). We create OpenBao ACL policies around these properties and let the authentication engine tie the two together, either explicitly or via groups.

In particular, permission grants (whether categorical in the case of user roles or granular in the case of direct user permissions) are given to a property of the authenticated entity: either the role they've been granted, some scope information (environment, branch, or later stage and job name), or the database id of the entity in question (usually only a user as jobs and pipelines lack a durable identifier across runs). This lets authentication and authorization via JWTs attest to the properties of the user (they have XYZ roles in this project and have user id 1234) or pipeline (executing on ABC branch in DEF environment of the project), which GitLab Rails knows and maintains in its existing databases. OpenBao then maintains the mapping of properties->permissions via its ACL policies.

While each potential property's policy is provisioned on a token, OpenBao ignores policies which don't exist in the system. This allows Rails to only create policies in OpenBao for which permissions are given. Rails can then query the list and contents of policies to display permission information, synthesizing that back via its databases to concrete roles, users, &c.

In this way, there's a separation of concerns: Rails handles the identity, but OpenBao keeps control over the ACL policies and the specific permission grants they entail. No cross-service synchronization of storage is required for this approach.

Proposed secret layout

We will use the existing secret layout.

The one caveat in this design is that "option 2" (wherein the same secret can appear with different names in different contexts) becomes hard to enforce; thus we'll use option 1 exclusively. While theoretically we can use the naming solutions, we can't easily validate that two paths have separate scopes when updating an ACL and thus will prefer to have separate names entirely. There is an approach to fix this in the notable restrictions section.

Ignoring this constraint and allowing name to be a strictly free-form value, forcing users to refer to secrets by some identifier we choose, could likely cause confusion and poor UX (and, doesn't solve the underlying issue reused

names to keep the production/staging difference as minimal as possible and enable the same `.gitlab.yml` file for both, just executing from different branches perhaps).

ACLs for pipelines

These ACLs would be used with the existing design's JWT auth methods.

One notable change is that, because we're relying on groups to associate permissions to pipelines (and no longer provisioning a per-execution pipeline JWT role and ACL policy), we will be able to use a single JWT role per project within the namespace, bound to GitLab's keys. This will allow us to directly issue JWTs and start pipelines without requiring a round-trip to OpenBao first.

ACLs for users

Based on current designs, we do not need to handle cross-project queries of any sort from users: they will be accessing only a specific secrets management page for a specific scope. Furthermore, our initial design only requires role-based access: granular permission access (in the future) would require us to solve the reverse-lookup problem, which is doable but requires an additional, more complex extension to OpenBao. However, custom roles (with granular scopes) are fully expressable in the new layout if they are named and can be queried by GitLab Rails to place on the JWT.

As before,

GitLab Rails will initially perform user operations on the back-end before we implement it in the front-end code. However, as GitLab Rails will not store a user access authorization table (but does have understanding of the user initiating the request and any roles they are granted in the project explicitly or implicitly through groups), we can continue using the property based approach with explicit sub-tokens if the roles conflict before policy unions land.

Note that OpenBao Proxy will not rewrite requests sent with an explicit token to use the global AppRole token. Use of direct token is not ideal long-term, but is a useful tool in this case as we can create expiring tokens that don't live much longer than the request they're used in, with less overhead than normal auth methods.

Types of operations

In order for this design to be effective, we need to know that every type of operation over secrets can be efficiently performed. We enumerate them below.

Pipelines fetch secrets

Pipelines must be able to fetch secrets from OpenBao. To fetch a secret:

1. GitLab Rails provisions a JWT, containing the concrete environment, branch, and project information including in a claim used for groupsclaim matching.
 - Initially GitLab Rails must query OpenBao to find the globbed groups directly, but if group aliases are updated to support globs, we can directly provision the final environments and OpenBao can match alias->globs internally.
2. GitLab Rails gives the JWT and the relevant paths to OpenBao to the Runner.
3. The Runner authenticates to OpenBao and fetches the secrets.

Steps 2 and 3 do not rely on any complex operations; only step 1 currently requires a LIST operation on the GitLab Rails->OpenBao side.

Observation: CI pipelines may default to static secrets, but most dynamic secret engines probably should have a type: keyword to allow the Runner to understand what type of operations to use.

User list secrets

User listing of secrets is limited to a particular space (project, ...) and thus incurs (initially) a single LIST operation. In the future, dynamic secret backends might complicate this, but we can likely use profiles to address that later.

Depending on the verbosity of the display, some information from viewing a specific secret might also apply and incur additional complexity.

User views specific secret

Viewing a specific secret implies we know the path in storage for the secret.

To view a specific secret, a LIST-PAGE call can be issued (size=1) to ensure the secret appears in the list. GitLab Rails can then fetch relevant ACL and metadata information using subsequent LIST operations against prefixed paths to get relevant policies. These ACLs will need to be READ to match secret to policy to understand relevant scope (based on the above).

In the future, we'll be able to skip the undirected LIST operation and all READ operations by using reverse lookup of policies; this will let us know (by virtue of policy naming) what things have access to this secret.

User modifies permission

To modify a permission, GitLab Rails would have to act on the user's behalf, fetching the relevant policy and then modifying it to have the new scope or removing an existing scope. The policy would either be updated (if additional paths remained in the policy) or it would be deleted. Any corresponding groups would have to be created or deleted as well.

Notably, while groups still exist on the entity (for any in-flight requests), this would have a real-time impact, unlike the current design, which generates ephemeral policies per-execution of a pipeline.

User sets value

To set a value, the user would perform a write operation to OpenBao (whether directly or through GitLab Rails acting on its behalf). Because the user already knows the path, this shouldn't impact anything.

Auditing user's access

Auditing a user's access to all secrets in OpenBao is fairly easy insofar as it is one or more LIST requests to relevant API endpoints based on project information. In particular, it requires a LIST operation to see if there are policies for the user's role and if the user has any directly given permissions (based on the policy bearing the user's id).

Across projects, this becomes harder: these lists must be done cross-tenant, which will be separate namespaces and thus won't share common LIST operations. We'll likely want to add separate background cleanup workers to the GitLab Rails user deletion, though as long as user ID is not ever reused, we should be fine leaving unused policies in OpenBao. Some longer-term expiry on permissions could also address this.

Deleting a project

Deleting a project requires deleting the relevant secret mount(s) and then listing and deleting all project-specific ACL policies and groups in OpenBao. A LIST operation can be used to see the list of policies under this project identifier and then remove all matching policies and/or groups.

Deleting a user

This roughly equates to the above (Auditing user's access), just with deletes being issued for any matching policies.

Timestamp computation

Since Rails will want to be ingesting and parsing OpenBao's Audit data anyways, we'll be able to maintain created, last modified, and last accessed, times. This could go into a table for more efficient lookup.

Notable restrictions

Uniqueness checks are hard

While we can support uniqueness within a given K/V engine, if we were to introduce hierarchical secrets in the future (at the group level for instance), it will be hard to ensure global uniqueness across group->project,

especially as users will have direct write capabilities in the future.

Thus, it is suggested to add an optional scope: tag to secrets (defaulting to project), to allow disambiguating where in the hierarchy this secret is expected to be read from. Secrets would then only need to be disambiguated across scopes.

However, except with a single unified K/V entry indicating the type, we'd also have uniqueness issues at the same scope for dynamic secrets. In that case, using a type: field (on the secret in the YAML file) will yield clarity (or potentially cause some confusion...). Using a single table has some benefits, insofar as allowing the runner to infer the type based on data stored in the K/V entry and allowing "option 2" based same-named secrets (in different scopes) to have different types associated with them (perhaps a static credential for testing but a dynamic value for staging and prod).

This is a future problem.

Descriptions and metadata

Not solved here is what to do with the description and other metadata fields (suggested rotation &c).

For K/V secrets, this can be a metadata value, but this will not work for non-K/V dynamic secrets. One option would be to provision a K/V entry for all values (as mentioned above): metadata fields for description could then be used and uniqueness across all types of secrets would potentially be enforced.

Theoretically this could also be done for inherited values (to "block" local engines from creating them), fixing the uniqueness check issue. However, it would not be ideal so using the scope: syntax would likely be preferable in that case.

Reused secret names

Concretely, for every secret, regardless of type (static vs dynamic), suppose we create the following entry:

- .../secrets/kv/metadata/explicit/<name>

This metadata entry could be used for existence checks (as KVV2 metadata entries show up in LIST operations under .../secrets/kv/data as well) and could hold GitLab-specific metadata about a secret.

Some metadata fields could include:

- type, to indicate the type of a secret (kv in the MVC for simple secrets).
- description, for the textual description of this secret.

- scopes, to indicate the known scopes of a secret. This could be a list such as:

```
json
[
  {
    "id": "<uuid>",
    "environments": [...],
    "branches": [...],
    "combined": [...],
    "stages": [...]
  },
  { ... }
]
```

to allow mapping from a single metadata name to an identifier per scope. Then, the actual (static) secret value could be at a path such as `.../secrets/kv/data/explicit/<name>/id/<uuid>` and the runner could fetch this value based on the desired match criteria. Similarly a scheme could be designed for dynamic secret engines, such as `explicit-<name>-id-<uuid>`.

- creator, for the identifier of the author of this secret.
- ... &c

This places a little more work on the runner, but we could design a migration procedure assuming all runners are updated to synchronize to the new GitLab version (or support landed there ahead of the GitLab Rails update with this). Migration could occur when a secret is modified to add an additional scope; this reduces the number of entries in OpenBao until such time as the same secret is required in different scopes. A UI could be built over this, to allow more intuitive creation/management of secret values. Everyone with some access to the secret would then be given read access to the metadata field.

OpenBao KVV2 supports a check-and-set semantic: this will allow us, without external transactions, to safely create and modify K/V entries.

Future extensions

The following extensions to OpenBao will allow us to improve performance and solve certain shortcomings with the existing design.

To summarize, only the two short-term extensions are required: if wildcards can be delayed until after MVC (or even after GA), then the ACL list prefix can also be delayed as well. This means this design should work without changes to OpenBao today, though with less performance if ACL listing is not implemented.

Short-term

The following short-term enhancements are required to be completed by GA as they're necessary just for the designs presented through MVC. They might have performance implications that allow us to reduce the work that GitLab Rails does.

ACL list prefix

ACL policies can contain the / character, which is still interpreted as an actual K/V path component separator. This means theoretically the LIST /sys/policies/acl could take an optional [/:prefix...] which corresponds to an on-disk subpath under the policy store, limiting results without having to use an iterated LIST-PAGE.

In particular, by prefixing our policies with a segment (project{id}/ or user/ or roles/ &c), we can list over just the policies applicable to each (project pipelines, or direct permissions given to users).

OpenBao already mostly supports this.

We will just need to support creating a SubView if ListPolicies(...) is given an optional prefix.

Complexity: simple

Impact: necessary performance improvement at scale

MVC: not strictly necessary; performance improvement

GA: required

Group alias glob matching

The identity subsystem

group aliases: alternative names (within one mount path) to which other names for specific groups apply. By enhancing these to support explicit globs, we can map glob policies to glob groups and have OpenBao apply them in the future.

This reduces GitLab Rails' from needing to query the exact set of groups and policies to apply before issuing the JWT in the first place.

To apply this mapping refreshExternalGroupMembershipsByEntityID will need to be updated to support globs in the groupAliases parameter. In particular, this is rather complex as the underlying MemDB does not support native glob matching. We'll need to create a new set